

DYNAMIC FAIRNESS MONITORING:

An Integrable MLOps Pipeline for Ethical Drift Detection

BRIDGEWATER STATE UNIVERSITY
Department of Computer Science
Spring 2026

Advisor: Dr. Haleh Khojasteh
Presented By: Frances Gallagher

ABSTRACT

Machine learning models deployed for high-stakes binary classification are susceptible to fairness degradation that performance monitoring infrastructures tend to deprioritize. Bias silently compounds alongside shifting data distributions, creating disparate outcomes across demographic groups that remain undetected until the magnitude of harm becomes statistically discernible. This capstone presents a demonstration MLOps fairness monitoring pipeline, designed to continuously evaluate inference batches in real time through four quantitative metrics: statistical parity difference, equal opportunity difference, predictive parity difference, and accuracy difference. By integrating automated preprocessing, scikit-learn's RandomForestClassifier, and a Flask-based REST API, this pipeline surfaces ethical drift through threshold-based alerting, rendering results on an interactive dashboard designed to be interpretable to both technical and nontechnical stakeholders. Two distinct datasets, UCI Adult Census and Kaggle Employee Attrition, were evaluated across three controlled bias simulations: gradual drift, oscillating drift, and bias-correction. Results demonstrate reliable drift detection, proportionate metric responses to varying bias intensities, and full recovery to baseline with no observed false positives. Metric evaluations identified equal opportunity difference as the most drift-sensitive and predictive parity difference as the least, across both datasets. Further findings support the conclusion that continuous fairness monitoring should serve as a prerequisite to deployment in any ML context. The pipeline's generalized architecture sustains consistent performance across binary classification domains without dataset-specific modification, requiring only a target column and protected attribute column as user inputs.

TABLE OF CONTENTS

INTRODUCTION..... 4

BACKGROUND DEFINITIONS.....5

SYSTEM ARCHITECTURE..... 9

a. DATA & PREPROCESSING..... 9

b. BASELINE MODEL & FAIR ASSESSMENT.....12

c. MONITORING PIPELINE ARCHITECTURE..... 14

BIAS SIMULATION.....17

VISUALIZATION..... 20

GENERALIZABILITY..... 23

EVALUATION RESULTS..... 25

TRADEOFFS & FUTURE MODIFICATIONS..... 28

ETHICAL IMPLICATIONS..... 29

REFERENCES..... 32

I. INTRODUCTION

Trained automated binary classification models drive high-stakes decisionmaking across industries, relying on statistical modeling to systemize baseline screening eligibility. Though effective at scale, such models cannot guarantee sustained ethical alignment through training data alone. Instead, fairness degrades dynamically alongside data shifts, allowing bias to emerge. ML models are susceptible to these drifts in ethical performance even when technically sound, making continuous fairness monitoring as critical as performance monitoring. Current solutions treat fairness as a static property evaluated only at deployment, leaving organizations unable to detect outcome gaps that widen over time across demographic groups, creating significant blindspots in anti-discrimination compliance. When undetected, these oversights can undermine customer trust and unknowingly place organizations in vulnerable positions.

Ethical drift has grown beyond a theoretical concern, and appears in documentation across domains. Consider a 2025 Stanford Report on generative AI use in resume generation, where such tools were observed generating resumes that listed fewer years of work experience for hypothetical older female applicants, and the opposite for the hypothetical male applicants [3]. Similarly, studies of deployed clinical decision support systems in English and Vietnamese hospital contexts identified outcome disparities across patient demographics only after deployment, by which patients had already been exposed to unequal treatment recommendations [4]. In each case, strong technical performance metrics masked the simultaneous degradation of model fairness, which remained undocumented. This gap between technical and ethical performance monitoring is what motivates the need for new continuous fairness monitoring infrastructures.

This capstone presents a systematic, dynamic approach to monitoring the fairness of deployed ML binary classification models to enable earlier detection of ethical drift before the damage compounds. This solution implements an integrable demonstration MLOps pipeline, which detects and surfaces unfair prediction patterns that would otherwise remain undetected within organization's existing monitoring infrastructures. By continuously tracking four predefined fairness metrics, and alerting when deviations surpass designated thresholds, this pipeline:

- I. Demonstrates a non-disruptive monitoring layer that logs batch predictions, monitors fairness metrics in real time, and alerts on statistically significant drift
- II. Establishes a statistical framework for distinguishing meaningful fairness degradation from natural metric variance across simulated data distribution shifts
- III. Visualizes drift effects through an interactive web-based dashboard accessible to both technical practitioners and nontechnical stakeholders
- IV. Validates that continuous fairness monitoring is a prerequisite for responsible model deployment, not an optional post-deployment consideration

This paper defines the theoretical and technical frameworks underlying each pipeline component, evaluates three automated drift scenarios across two datasets, and grounds all findings in a broader analysis of ethical risk in deploying AI for high-stakes decisionmaking. It is important to note that this pipeline serves as a proof of concept, with future modifications and scope expansions discussed in the concluding section.

II. BACKGROUND DEFINITIONS

Establishing the theoretical framework underlying this pipeline is necessary before describing its system architecture. The key concepts to consider are algorithmic fairness, protected attribute, statistical parity, equal opportunity difference, predictive parity, accuracy difference, and data drift.

Algorithmic fairness refers to a model's property of producing outputs that do not systematically disadvantage any one protected group over another. Fairness is distinct from *accuracy* and can be analyzed at both the individual and group level. A technically sound model is not inherently unbiased, as fairness and accuracy are orthogonal properties that must each be independently evaluated. A resume-screening model could achieve high accuracy for one demographic group while producing significantly worse outcomes for another, while a loan-approval model could maintain equal selection rates across groups yet produce systematically incorrect predictions for both. This pipeline treats accuracy and fairness as equally necessary conditions for responsible deployment, operating on the premise that optimizing one at the expense of the other increases a model's risk of producing biased outcomes.

A *protected attribute* is any demographic characteristic (e.g. sex, race, age, disability status) that anti-discrimination laws prohibit from weighting unfairly in decisions. Such attributes, though not direct model inputs, can influence predictions indirectly through proxy variables. In cases where training data encodes historical patterns, variables such as `first_name` for sex or `zip_code` for race cause models trained on such data to reproduce the same systemic biases. To combat this risk, protected attributes in the datasets evaluated by this pipeline are represented as binary user-specific features, allowing for flexible integration across domains. Privileged and unprivileged groups are therefore auto-detected through alphabetic sorting of unique attribute values. Removing the protected attribute columns entirely would not eliminate proxy-based bias, and would prevent the pipeline from computing the group-level fairness metrics necessary for monitoring.

As bias poses a dynamic threat, no single metric can capture its full effect on a model's outputs. Therefore four metrics were selected to collectively characterize fairness across the pipeline: statistical parity difference, equal opportunity difference, predictive parity difference, and accuracy difference. Across all metrics, negative values suggest disadvantages for the unprivileged groups relative to the privileged group, while positive values indicate a relative advantage, and near-zero values indicate approximate parity. The following four metrics are not independent of one another, as optimizing for one metric may structurally limit performance on another, which reinforces the need for concurrent monitoring, rather than evaluating any single metric in isolation.

- a. *Statistical parity* is calculated by subtracting the privileged group's positive prediction rate from that of the unprivileged group, measuring whether favorable outcomes are distributed at comparable rates across groups. An example within loan approval could output 40% approval for female applicants and 70% for male, with the statistical parity difference of $0.40 - 0.70 = -0.30$, indicating that female applicants receive favorable outcomes at a significantly lower rate. The ideal parity difference is zero, reflecting equal selection rates across groups. It is important to note, however, that a value of zero does not guarantee equitable treatment at the individual level or across all outcome types.
- b. *Equal opportunity* measures whether the model identifies qualified individuals at comparable rates across groups, regardless of overall selection rate. The formula subtracts the privileged group's true positive rate (true positives divided by total actual positives) from that of the unprivileged group. Among applicants who genuinely qualify for a positive outcome, if the model approves only 60% of women and 85% of men, the equal opportunity difference is $0.60 - 0.85 = -0.25$, indicating that qualified women are systematically missed at a higher rate. The ideal difference is zero, and unlike statistical parity, equal opportunity difference is particularly sensitive to how well the model performs specifically among those

who should receive favorable outcomes. This suggests equal opportunity is a stronger indicator of discriminatory omission than overall selection rate.

- c. *Predictive parity* measures the reliability of positive predictions across groups. It is calculated by subtracting the privileged group's precision (true positives divided by total predicted positives) from the unprivileged group's precision. If the model's positive predictions are correct for only 70% of female applicants but 90% of male applicants, the predictive parity difference is $0.70 - 0.90 = -0.20$, indicating the model's approvals are less reliable for female applicants. It is important to note that an ideal zero predictive parity difference alongside a zero statistical parity difference is only achievable when the base rates of positive outcomes are equal across groups.
- d. *Accuracy difference* measures overall correctness of predictions across groups, and is determined by subtracting the accuracy of the privileged group from that of the unprivileged group. Though the most straightforward of the four metrics, it does not distinguish false positive and false negative errors, making it insufficient as a standalone fairness indicator. Similar to each previous metric, the ideal accuracy difference is zero, with negative values indicating lower predictive accuracy for the unprivileged group relative to the privileged group, and positive values indicating the inverse.

A model's statistical properties are not inherently static values, but rather change with time, degrading model performance through a process known as *data drift*. This data drift has the potential to degrade each metric gradually and covertly, in a way that maintains overall model performance while simultaneously widening disparities between groups. Two principal forms are relevant here: *covariate drift*, in which the distribution of input features changes over time, and *label drift*, in which the statistical relationship between features and outcomes changes. Both can silently skew fairness

metrics without triggering standard performance error alerts, a discrepancy that this pipeline is designed to address. By the time drift becomes visible in aggregate accuracy, its effects have typically already compounded across many predictions.

III. SYSTEM ARCHITECTURE

To continuously monitor model fairness across production inference, the pipeline evaluates metrics through *batches*, defined as individual complete end-to-end prediction runs on the established fairness evaluation set. To identify how a model's fairness profile deviates over time, the pipeline's architecture is structured around three key components: preprocessing, baseline model training, and continuous monitoring. It is designed to maintain isolated predictions to prevent overlapping runs, while simultaneously displaying interpretable alerts to users through an interactive web-based dashboard.

```
class FairnessConfig:
    def __init__(self, df, target, protected_attr, desired=None, privileged=None):

        # auto-detect desired & privileged labels (binary 0/1, auto-selects second val alphabetically)
        if desired is None:
            vals = sorted(self.df[target].dropna().unique())
            self.desired = vals[-1] # ex: 0/1 for >/< 50k
            print(f"auto-detected desired outcome: {self.desired}")
        else:
            self.desired = desired

        if privileged is None:
            vals = sorted(self.df[protected_attr].dropna().unique())
            self.privileged = vals[-1] # ex: 'male', 'female'
            print(f"auto-detected privileged group: {self.privileged}")
        else:
            self.privileged = privileged
```

**Figure A, FairnessConfig label auto-detection*

a. DATA & PREPROCESSING

Users have the ability to upload any .csv or .data file applicable to binary classification through the UI, requiring only two manual entries: the target

column and the protected attribute column. The remaining configuration is fully automated through a `FairnessConfig` class (refer to **Figure A**), which handles dataset-specific preprocessing, sorting, and labeling without any additional user interaction.

`FairnessConfig` scans the unique values of the user-specified target and protected attribute columns through alphabetical sorting, selecting the last value as the desired positive outcome and privileged group designation, respectively. This eliminates the need for manual label specification and produces consistent results across datasets. `FairnessConfig` then partitions feature columns into numeric (`int64` and `float64` dtype) and categorical (`object` dtype) subtypes for downstream processing. In cases where alphabetic ordering does not yield the intended designation, the desired and privileged labels can be passed as optional constructor arguments to override auto-detection. The target and protected attribute columns are explicitly excluded from the feature set to prevent the model from directly accessing group membership or outcome labels during training. The protected attribute is later retained as a separate binary array solely for fairness metric computation, never as model input.

Each key parameter receives a designated binary label, with `target=1` only when values equal desired outcomes, while `protected_attr=1` represents the privileged group and `protected_attr=0` represents the unprivileged group. This is vital when considering fairness calculations, as metrics computed downstream are designed to rely on binary representations. Missing values are handled at two levels: at the dataset level, placeholder values are replaced with NumPy NaN prior to configuration, while at the feature level, the pipeline applies imputation within each transformer branch. Scikit-learn's `ColumnTransformer` is fit and applied to

the full feature set (refer to **Figure B**), with the resulting preprocessed arrays passed to the train/test partitioning step. The numeric branch uses mean-based imputations, utilizing a SimpleImputer alongside a StandardScaler to normalize features to zero mean and unit variance. Standardizing features prevents multiple scales from influencing model training, and ensures all numeric inputs provide

```
# create preprocessor pipeline
numeric_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='mean')),
    ('scaler', StandardScaler())])

categorical_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='most_frequent')),
    ('encoder', OneHotEncoder(drop='first', sparse_output=False,
                              handle_unknown='ignore'))])

preprocessor = ColumnTransformer(
    transformers=[
        ('num', numeric_transformer, numeric_cols),
        ('cat', categorical_transformer, categorical_cols)])

X_processed = preprocessor.fit_transform(X) # fit & transform
```

**Figure B, preprocessor pipeline*

proportionate contributions. The categorical branch utilizes frequency imputation with SimpleImputer, and encodes each feature with OneHotEncoder. The OneHotEncoder is configured with drop='first' to reduce dimensionality by removing one redundant dummy variable per categorical feature. The handle_unknown='ignore' setting ensures that any category values present in the evaluation or test sets, but absent from training, are silently ignored rather than causing transformation errors. Such measures must be taken to ensure a clean output of fully processed NumPy feature arrays, alongside binary target and protected attribute arrays, globally stored for reapplication.

Once processed, the dataset is stratified into distinct groups over two splits. The first split divides a temporary working set from the dedicated fairness set, producing an 80/20 split on the target variable specifically for distribution preservation. The second split is applied to only the 80% working data, further partitioning it into training and testing sets, with a 75:25 ratio yielding

approximately 60% train and 20% test relative to the entire dataset. To preserve performance integrity and validity of monitoring assessments, the fairness evaluation set is deliberately separated from both training and testing data, and rebalanced once more to ensure fair distribution. Privileged and unprivileged groups are indexed separately, downsampling each to match the size of the smaller group, ensuring equal representation. The combined indices are shuffled once a fixed random seed is defined to remove ordering artifacts and ensure reproducibility. This balance is critical, as an unequal group distribution in the evaluation set would systematically skew metric calculations and produce a misrepresented baseline. The resulting fairness evaluation subset is returned to the upload endpoint, and stored in server memory for use across subsequent batch inference calls.

b. BASELINE MODEL & FAIR ASSESSMENT

The key working component of this pipeline is the RandomForestClassifier (refer to **Figure C**), a supervised machine learning algorithm that aggregates multiple decision trees. RandomForest was selected for its mixed feature handling, structural robustness to noise, and built-in feature importance scoring, which provides a basis for explaining prediction inconsistencies beyond the statistical outputs. As preprocessed datasets contain a combination of scaled numerical and encoded categorical features, it was necessary to train a classifier capable of handling varying feature types without additional transformation. Similarly, evaluating across multiple datasets increases exposure to noise accumulation, making the ensemble structure's robustness to outlier influence particularly valuable. This allows for more thorough and accurate representations of model behavior in instances of randomized data flow.

The model is configured with `n_estimators=200` and a fixed random seed throughout, training and aggregating predictions across 200 decision trees. A higher tree count improves prediction stability through ensemble averaging, at the cost of increased training time. Maximum tree depth is bounded at 15 to prevent overfitting, constraining the model from memorizing training patterns that do not generalize. The minimum samples required for splits at internal nodes and leaf nodes are set to 10 and 5 respectively,

```
# shuffle idxs
np.random.shuffle(fairness_idx)

# create fairness eval set
X_fair = X_fair[fairness_idx]
y_fair = y_fair[fairness_idx]
protected_fair = protected_fair[fairness_idx]

# random forest classifier initialized w balanced weights
model = RandomForestClassifier(
    n_estimators = 200, # tree num
    max_depth = 15, # overfitting
    min_samples_split = 10,
    min_samples_leaf = 5,
    class_weight = 'balanced',
    random_state = 42,
    n_jobs = -1,
    verbose = 1)

# train model
model.fit(X_train, y_train)
```

**Figure C, RandomForestClassifier*

providing an additional layer of regularization that prevents the model from partitioning on insufficiently small subsets. `class_weight='balanced'` serves as an ethically critical parameter for this classifier. High-stakes classification domains such as income prediction and employment screening utilize inherently imbalanced datasets, where the majority class typically corresponds to negative outcomes. Training on imbalanced data would optimize outputs for the majority class, producing accuracy that is only meaningful for the privileged group. The balanced configuration automatically increases the penalty for misclassifying the minority class, ensuring both outcome classes contribute proportionately to model training. A balanced configuration is nonnegotiable, as unbalanced configurations carry substantial underperformance risk for already

underrepresented groups, making class balancing an ethical prerequisite rather than an optional tuning choice.

The fairness baseline is established by running inference on the balanced evaluation set with zero bias injection. The resulting predictions are then passed through the fairness metric calculator function and globally stored as monitoring references, returning four difference values: statistical parity, equal opportunity, predictive parity, and accuracy. These baseline values represent the model's behavior on balanced, clean data, identifying natural disparities between groups present in the training data distribution. Initial metric deviations are expected, reflecting real-world data variation. As an example, a baseline statistical parity difference of -0.3350 indicates that the unprivileged group receives positive predictions at a rate 33.5 percentage points lower than the privileged group, under zero bias injection. This reflects historical income inequality present in the census data, not a modeling error. All subsequent metric calculations track deviations from this baseline in both positive and negative directions, flagging degradation relative to the model's established clean-data behavior.

c. MONITORING PIPELINE ARCHITECTURE

Data surveillance lies at the core of the pipeline, accepting batches, running inference, and surfacing degradation in real time. With each component designed around reproducibility, the model itself remains untouched and only monitored.

As a Flask-based system, the backend exposes three endpoints to communicate between the ML components and the UI. The `/upload` endpoint handles initial configuration, triggering the preprocessing and training pipeline to compute and store the baseline fairness metrics. The `/run_batch` endpoint receives a bias

configuration from the frontend, optionally applies prediction-level bias injection, runs `predict()` and `calc_fairness()`, and returns the resulting metrics as a JSON response. The `/health` endpoint ensures that batch operations can only begin given an initialized backend with a fully trained and configured model loaded. Flask-CORS is enabled consistently across all endpoints to permit cross-origin requests between the frontend served on port 8000 and the backend running on port 5000.

The fairness evaluation set constructed at training serves as the fixed batch input, a choice made deliberately to ensure any metric change is solely attributed to bias injection rather than data variation. When bias injection is enabled, `inject_bias()` is applied to the model's output predictions prior to fairness calculation. Rather than modifying input features, the function selectively flips a proportion of positive predictions to negative for the unprivileged group, with the proportion determined by the intensity parameter. This prediction-level approach provides a dataset-agnostic simulation of disparate impact without altering the stored evaluation arrays, ensuring that batches restore clean inference once injection is disabled.

To detect violations, the frontend compares each batch's returned metric values against their corresponding baseline values. Deviations beyond an absolute threshold of 0.05 trigger system alerts, with overall status ranging across stable, warning, and critical. This threshold value was selected to filter out naturally occurring variance that appears at minor levels between batches regardless of data cleanliness, while remaining sensitive enough to flag meaningful early drift. Flagged violations log a batch identifier, metric label, and current value alongside the updated system status indicator. Alert status is determined by two factors

evaluated together: the count of violations across the most recent five batches, and the cumulative ratio of biased to clean batches across the full run history. Evaluating both prevents a single clean batch from prematurely resetting a critical alert following sustained degradation, reducing the risk of false negatives in status reporting.

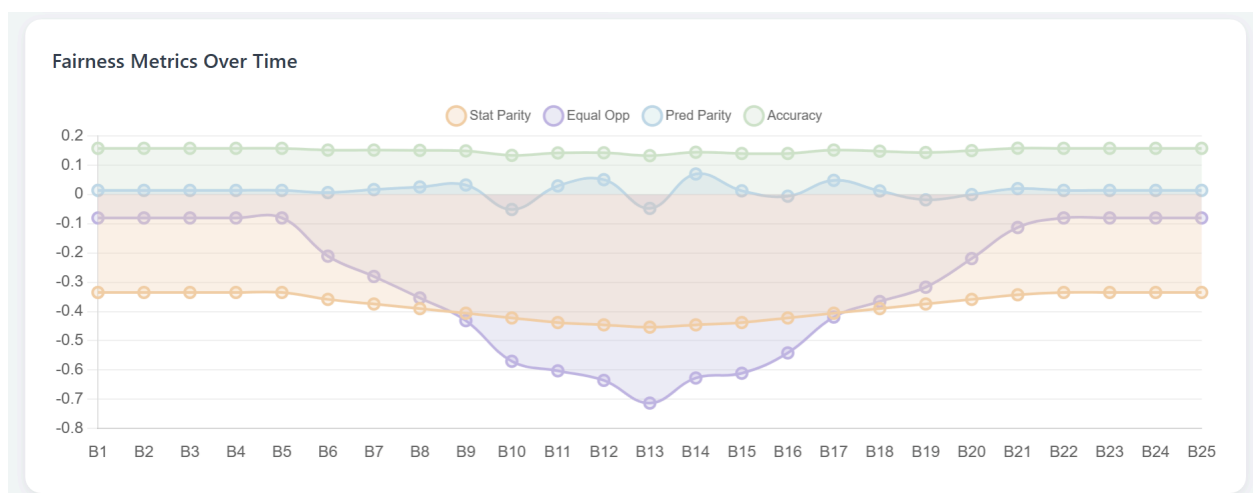
Global variables are used to persist the trained model, preprocessing pipeline, and evaluation arrays between batch requests. This method is deliberately utilized within single-user demonstration contexts, as concurrent requests could overwrite shared state, producing array dimension mismatches between the stored evaluation set and the model's expected input. Replacing global state with proper session management is identified as a priority future modification.

The three architectural components of the monitoring pipeline form a continuous surveillance loop that prioritizes configurability and transparency. By separating preprocessing, model inference, and fairness evaluation into discrete, independently accessible stages, the pipeline remains extensible without requiring architectural changes to any single component. This design ensures usability across proficiency domains, and positions the pipeline as a functional prototype for a class of monitoring infrastructure that remains largely absent from standard ML deployment practices.

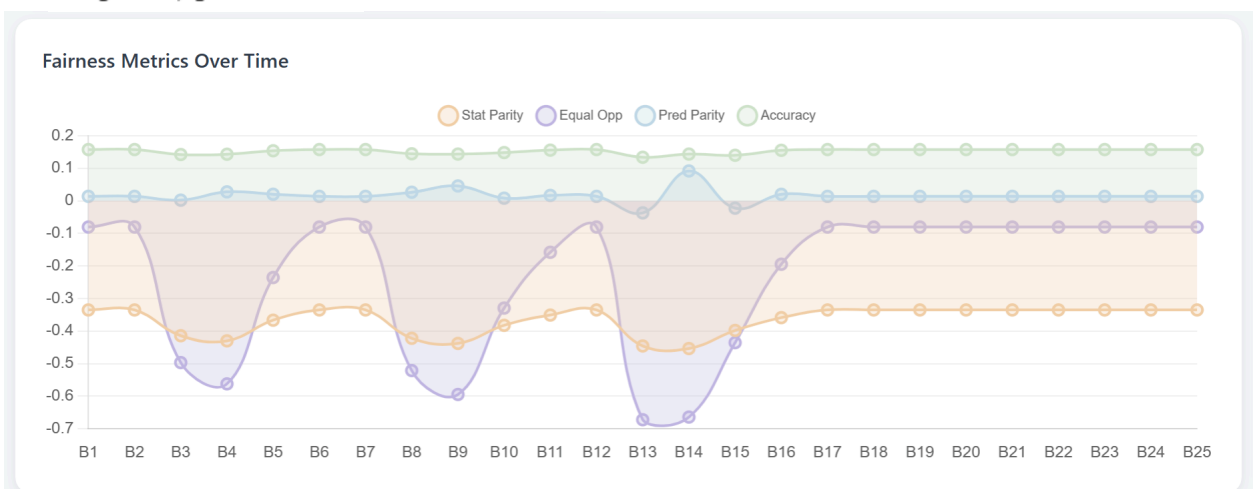
IV. BIAS SIMULATION

As fairness drift cannot be inherently identified during integration testing, it could be argued that simulations should be considered prerequisites to deployment. Simulations allow for end-to-end system audits while utilizing feature degradation as a proxy for real-bias, which can emerge from unclean data collection, deprecated variables, or biased feedback loops. To recreate, the `inject_bias()` function was implemented, directly

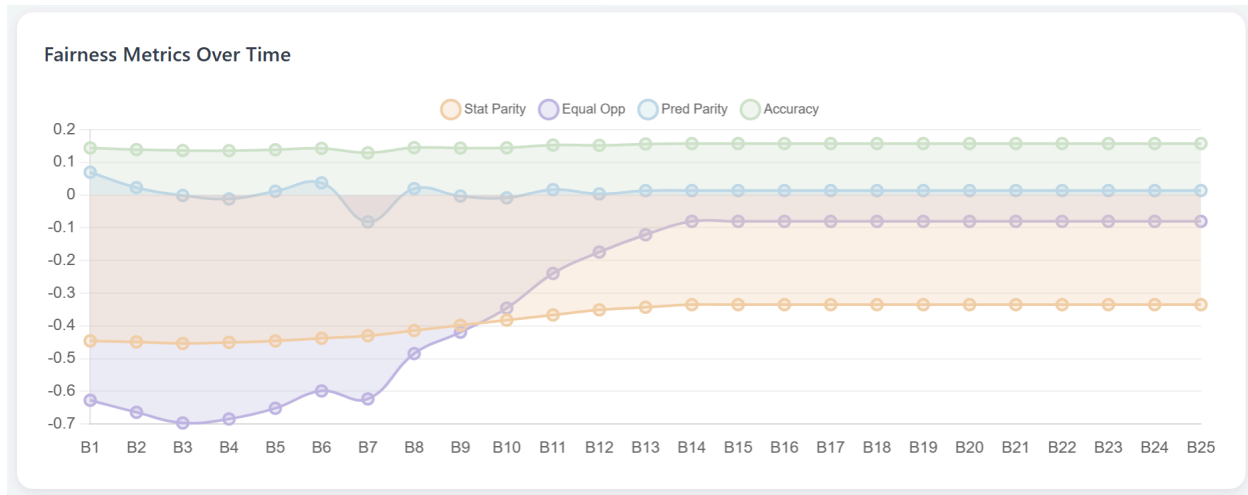
flipping a proportion of positive predictions to negative for the unprivileged group based on a user-specified intensity parameter. Altering predictions directly serves as a dataset-agnostic degradation simulation, maintaining input features while modifying outputs. The key component lies in that the model does not retrain between batches, allowing all observed metric deviations to be solely attributable to the injected bias, not to model retraining or parameter updates. The `inject_bias()` function was then applied across three contrasting simulations representing real-world edge cases: gradual bias introduction, intermittent (oscillating) bias introduction, and a bias-correction simulation. Batch-level injection intensities were predetermined across all twenty-five batches for each scenario as follows.



**Figure D, gradual bias*



**Figure E, oscillating bias*



**Figure F, bias-correction*

Simulating gradual bias (reference **Figure D**) most closely mirrors the uncontrolled environments that breed the most potent fairness degradation. This form appears at a rate either completely undetected or significantly challenging to identify, and typically occurs as feature distributions shift alongside changing population demographics or as training data becomes obsolete. To simulate this, prediction flipping intensity is gradually increased then reduced over twenty-five batches, ranging from 0% to 75% intensity. The first five batches remain at 0% injection, representing a stable post-deployment period. Bias injection begins at 15% intensity starting at batch six, increasing incrementally through batches twelve and thirteen and peaking at approximately 75% intensity. Batches fourteen through twenty-five decrease at the same rate until returning to 0%, forming a pattern designed to test whether the pipeline flags early-stage drift before it reaches peak intensity. The gradual scenario is particularly valuable for metric evaluation, as it identifies the location, specific metric, and intensity level at which threshold violations first occur.

Modeling oscillating bias (reference **Figure E**) introduces injection intermittently rather than as sustained drift. Though less common in practice, such patterns have the

potential to emerge in deployments characterized by variability, inconsistency, and environment-specific shifts, such as overlapping release cycles or conflicting asynchronous updates. This is simulated through rapid bias injections and removals across windows of two to three batches, beginning with a spike to 50–60% intensity in batches three and four, immediately followed by a return to 0%, and then reinjected dynamically across two additional windows with peak intensities of 50%, 65%, and 75% respectively. Such irregular patterning intentionally places varying gaps between injection events, testing the pipeline's capability to distinguish true degradation from statistical noise. This approach tests whether the system incorrectly triggers false positives across inconsistent severity ranges, while simultaneously evaluating its sensitivity to intense drift. This simulation most directly correlates with the Fairness Metrics Over Time and Batch History dashboard components, as both visualize the location and intensity of intermittent bias across the full batch history.

The final bias-correction simulation (reference **Figure F**) tests the complete operational cycle this demonstration pipeline is designed to support: detecting bias in a preexisting model, flagging said bias, and visualizing its predicted behavior once manually repaired. This is achieved through high-intensity preliminary injections of 70–75% throughout the first five batches, representing a pre-biased model state. Injections then decrease to 0% across the following eight batches, with the remaining batches sustaining stable 0% injection. Bias does not return after batch fourteen, with the correction phase assuming successful human intervention has resolved the underlying source of drift (e.g. manually rebalanced training set). Unlike the gradual and oscillating scenarios, the bias-correction simulation begins at maximum drift intensity, testing the system's ability to both detect severe baseline deviations and track metric recovery as injections are progressively reduced. This scenario outlines the number of batches required for all four metrics to

return to stable status, and demonstrates how system alerts are expected to progress from critical, to warning, to stable across a full correction lifecycle.

Across the three controlled scenarios, statistical parity and equal opportunity difference were consistently triggered at lower intensities, while shifts in predictive parity and accuracy difference were only detected at higher intensities. This suggests each metric carries a distinct sensitivity profile relative to injection intensity, indicating that the pipeline has potential to identify a degradation hierarchy revealing which fairness dimensions are compromised first.

V. VISUALIZATION

The foundation of this pipeline lies in creating accessible means to evaluate the fairness of preexisting ML binary classification models. The UI presents a comprehensible and interactive dashboard structured to allow for general system triage before deep analysis. Live updates reflect system statuses alongside batch runs, categorized through a tiered color system, ranging from green to beige to purple, indicating increasing levels of drift. Graduated alert visuals were deliberately chosen over harsh color contrasts to prevent alert fatigue, prompting attention proportionally to drift magnitude rather than inducing alarm at low severity levels. The following details key components of the dashboard UI.

The first tier of the dashboard presents the highest-level system status indicators, designed to direct immediate attention to the metrics most sensitive to data shifts and emerging bias. The activity feed logs individual metric violations in real time, surfacing which batches and metrics triggered alerts most recently. The color-coded system status card reflects the pipeline's overall fairness state, shifting between STABLE, WARNING, and CRITICAL based on both the proportion of clean batches across the full run history and the number of violations detected within the most recent five batches. A STABLE designation requires that at least 70% of all processed batches remain within threshold,

and that no violations appear in the most recent five batches. WARNING is assigned when either condition is partially violated, and CRITICAL when fewer than 30% of batches are clean or four or more of the five most recent batches contain violations.

The dashboard then centralizes tracking fairness metrics over time, displaying stats for each individual metric through a multivariate line chart. Each line represents a single metric's trajectory across all processed batches, presenting key divergence trends by locating the magnitude and direction of deviation, identifying the order in which metrics degrade, and visualizing their sensitivity to bias correction. A primary analytical value of this chart is its capacity to surface cross-metric correlations (e.g. statistical parity degrading before equal opportunity), as such patterns would be obscured in single-metric or tabular visuals.

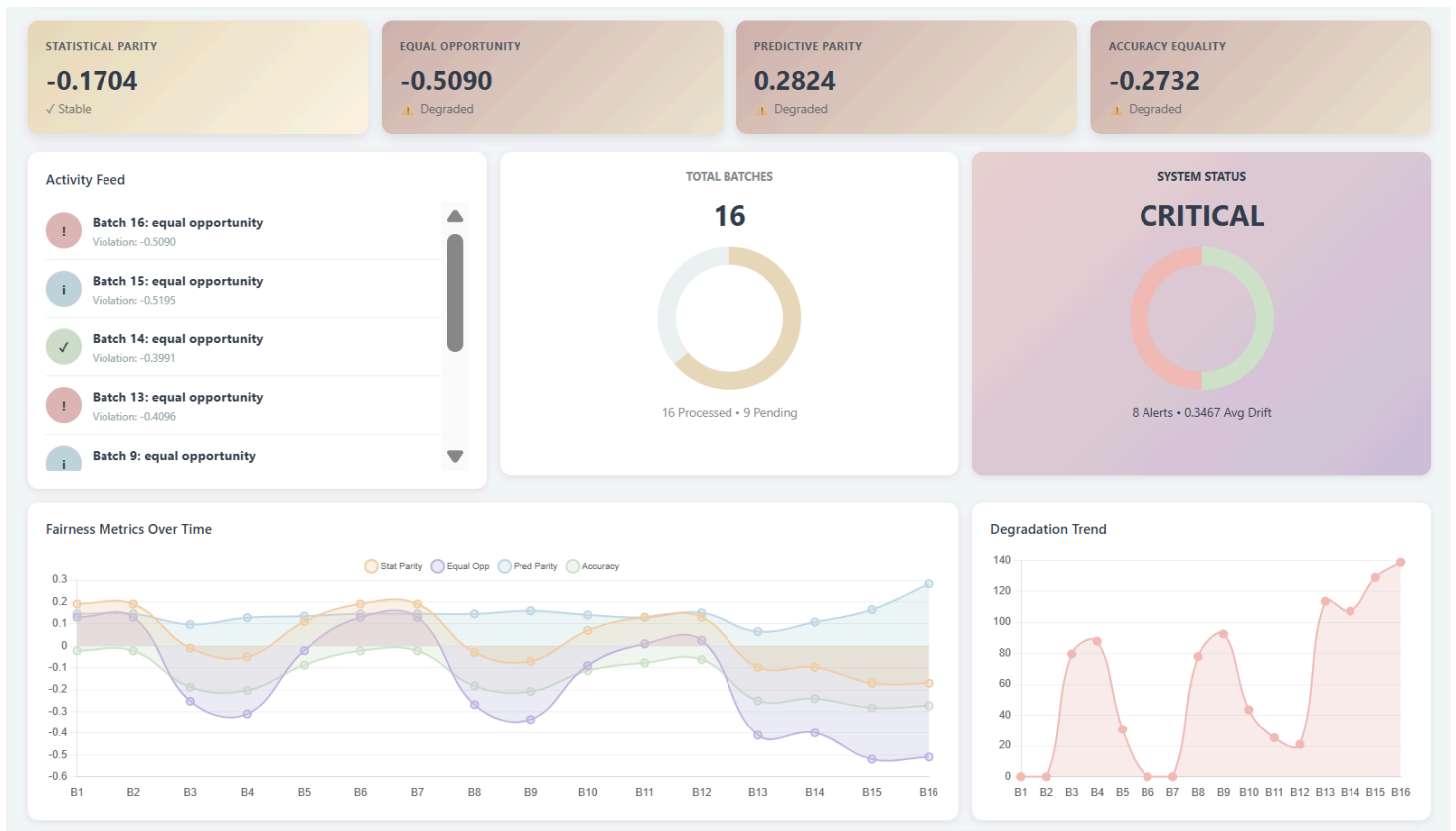
The Degradation Trend chart is the secondary most diagnostic component, providing the composite sum of absolute baseline deviations across all four metrics rather than tracking each individually. Combining these scores into a single trend line answers the question of whether the model's fairness profile is improving or declining across batch runs. The shaded region expands as aggregate drift increases, providing an immediate visual indicator of acceleration. This chart complements the Fairness Metrics Over Time chart by providing a compressed summary of how individual metric trajectories collectively contribute to overall bias levels, useful for practitioners who require a high-level signal before investigating individual metric breakdowns.

The Batch Comparison table provides the numerical data the trend charts represent, by providing full metric breakdowns for recent batches alongside a color-coded fairness badge. It surfaces key numerical data not otherwise displayed from the visual analyses alone, communicating precise values for technical practitioners to refer to. Reverse

chronological ordering places the most recently impacted batches at the highest priority for review. Fairness badges flag any batch in which one or more of the monitored metrics exceed the baseline deviation threshold, receiving either a green or pink depending on violation status. Providing exact values, the comparison table provides a technically-oriented visualization for stakeholders requiring in depth numerical analysis.

Finally the Batch History Grid directly complements the metric analysis by identifying which specific batches are most heavily impacted, with continuous color shading communicating severity at a glance. All twenty-five batches are represented as individual blocks, with color determined by the average deviation magnitude across all four metrics on a continuous gradient scale: grey for unprocessed batches, green for batches near baseline, transitioning through light beige and warm beige for moderate drift, and into a darker purple/red for severe deviation. This continuous gradient was chosen over discrete violation counts to more accurately reflect the proportional nature of fairness degradation. The grid was designed to facilitate pattern recognition, allowing an immediate full-run overview that identifies where bias emerges, how long it persists, and how quickly the system recovers. Alongside the three automated simulation scenarios, the history grid serves as a single-visual summary of each scenario's complete drift and recovery trajectory.

The use of these visualizations were designed to facilitate metric comprehension between both technical and nontechnical stakeholders, prioritizing accessibility alongside depth. The deliberate ordering from high-level status indicators numerical analysis provides an immediate triage entry point before directing users toward deeper investigation. This layered structure ensures that fairness degradation is not only detectable, but identifiable and communicable across proficiency levels.



*Figure G, interactive dashboard

VI. GENERALIZABILITY

In terms of this pipeline, *generalizability* refers to the system's capacity to process and evaluate a variety of classification datasets without data-specific code modifications. Since initial development, the core framework was designed as a tool applicable to any ML-based binary classification model, intentionally abstracted to ensure compatibility with relevant labeled datasets. This was primarily achieved through automated runtime processes that demonstrate consistent adaptability across datasets while maintaining predictive performance or fairness evaluation integrity. Real-world data flow is not domain-specific, therefore requiring universal monitoring infrastructures.

Though the classifier was trained on the US census adult and employee datasets, each variable and method was designed to accommodate variation across sets. Labels are autodetected through alphabetic sorting rather than hardcoding, selecting the last value in sorted order as the desired positive outcome. As an example, the Adult Census income column contains the values ' $\leq 50K$ ' and ' $> 50K$ ', so alphabetic sorting selects ' $> 50K$ ' as the desired outcome, as its leading character has a higher sort order. Feature columns are dynamically separated from identified attributes through pandas select_dtypes inference, and renamed to col_0, col_1...col_n to ensure identical data handling while avoiding pipeline failures. Preprocessing is performed on a ColumnTransformer pipeline, autodetecting column amounts and applying both StandardScaler and OneHotEncoding on those applicable. Mismatched features are handled by default, allowing fairness evaluation to operate through binary encoded arrays (y_pred, protected), such that any CSV file, target column name, and protected attribute column name can be passed as arguments to initialize the pipeline.

Two structurally distinct datasets were used to demonstrate versatility: adult.data and employee.csv, taken from UCI Machine Learning Repository and Kaggle respectively. The Adult Census dataset consisted of approximately 32,561 rows and 14 features, with its identified target as income ($\leq 50K$ / $> 50K$), and protected attribute as sex (male/female, operationally). It primarily serves within a socioeconomic domain, and consists of a wide variety of numeric and categorical features. The Employee Attrition dataset conversely has approximately 5,000 rows and 9 features, with its identified target as LeaveOrNot (stayed/left), and protected attribute as gender (man/woman, operationally). This exists within the workforce domain, containing fewer features overall, with heavier proportions of categorical variables over numeric ones. The end-to-end preprocessing, model training, fairness evaluation, and dashboard output remain consistent, displaying data deviations from respective baselines as expected. Metric differences were

predominantly rooted in contrasting starting values, reflecting demographic patterns within domains rather than inconsistency.

While additional sets were not evaluated due to time constraints, the successful end-to-end analysis of two distinct datasets supports the conclusion that this pipeline generalizes to any labeled binary classification dataset without dataset-specific modification. It demonstrates a versatile architecture serving as a proof of concept that a universal approach allows for more thorough fairness evaluation than bounded implementations. It is important to note that generalizability in this case is contingent on the presence of a binary target, and a protected attribute that can be expressed as binary group designation.

VII. EVALUATION RESULTS

Both the Adult Census and Employee Attrition datasets were evaluated across three bias simulation scenarios: gradual, oscillating, and bias-correction. This evaluation tests the reliability of the pipeline's detection mechanisms and the generalizability of its architecture. All metrics are evaluated relative to each dataset's established baseline rather than an absolute zero standard, as the pipeline is designed to accommodate naturally varying metric trajectories.

	Adult Census	Employee Attrition
Statistical Parity Diff	-0.3350	+0.1905
Equal Opportunity Diff	-0.0804	+0.1297
Predictive Parity Diff	+0.0139	+0.1457
Accuracy Diff	+0.1576	-0.0226

**Figure H, baseline metrics*

Figure H presents the baseline fairness profiles for both datasets across all four evaluation metrics. The Adult Census dataset produces a baseline statistical parity difference of -0.3350, indicating the unprivileged group receives favorable predictions at a significantly lower rate than the privileged group. This negative value reflects real-world income inequalities present in the census data. The equal opportunity difference of -0.0804 sits considerably closer to zero, indicating the model represents qualified individuals across groups with relative consistency. The predictive parity difference of 0.0139 suggests near-equal precision across groups at baseline, while the accuracy difference of 0.1576 indicates the model performs meaningfully better for the unprivileged group in overall correctness, a pattern attributable to the class imbalance corrections applied during training .

The Employee Attrition dataset presents a contrasting profile, with three of four baseline metrics in the positive range. Statistical parity at 0.1905, equal opportunity at 0.1297, and predictive parity at 0.1457 all indicate a baseline advantage for the unprivileged group, directly opposing the Adult Census profile. The accuracy difference of -0.0226 remains the only negative metric and sits well within the alert threshold. This bidirectional contrast confirms that the pipeline correctly monitors degradation from both negative and positive baselines without modification, a key requirement for generalizability.

Figure I summarizes detection latency, peak equal opportunity deviation, and recovery batch across all three scenarios for both datasets, and will be referenced in analysis:

	Adult Census			Employee Attrition		
	First Alert	Peak Equal Opp	Recovery	First Alert	Peak Equal Opp	Recovery
Gradual	Batch 6	-0.7131	Batch 22	Batch 6	-0.3991	Batch 22
Oscillating	Batch 3	-0.6845	Batch 17	Batch 3	-0.4148	Batch 17
Bias Correction	Batch 10	-0.3539	Batch 14	Batch 1	-0.3991	Batch 12

**Figure I, key data points*

In the gradual scenario, bias was introduced incrementally beginning at batch 6, peaking at batch 13, and recovering to baseline by batch 22 in both datasets. Equal opportunity difference was the first metric to trigger alerts in both cases, as flipping positive predictions directly reduced its true positive rate. In the Adult Census dataset, equal opportunity difference declined from its baseline of -0.0804 to a peak of -0.7131 at batch 13, a deviation of approximately twelve times the alert threshold. The Employee Attrition dataset produced a peak equal opportunity difference of -0.3991, representing a deviation of approximately ten times the threshold from its baseline of 0.1297. Both datasets achieved full recovery by batch 22, confirming the pipeline successfully clears alert states once injection ceases.

In the oscillating scenario, three bias windows of increasing severity were introduced between periods of zero injection, with both datasets triggering first alerts at batch 3. The Adult Census dataset produced a peak equal opportunity difference of -0.6845, while Employee Attrition peaked at -0.4148. The pipeline returned to baseline between each injection window with no false positives and no carryover of alert state between bias events. Peak deviations escalated proportionally across the three windows in correspondence with increasing injection intensity, demonstrating the pipeline's sensitivity to drift magnitude. Both datasets recovered cleanly by batch 17.

In the bias-correction scenario, the dataset begins at a critical bias state and simulates the pipeline's behavior following human intervention to resolve drift. Detection latency differed substantially between datasets: Employee Attrition triggered an alert immediately at batch 1, while Adult Census did not trigger until batch 10. This discrepancy reflects the difference in baseline magnitudes, as the Adult Census statistical parity baseline of -0.3350 requires a larger absolute deviation to exceed the fixed threshold of 0.05 than the Employee baseline of +0.1905, structurally reducing early detection sensitivity. Employee Attrition achieved full recovery by batch 12, while

Adult Census required until batch 14. Both datasets reached full baseline recovery, confirming the pipeline's ability to track the complete correction lifecycle.

Across all three scenarios, the pipeline reliably detected fairness degradation in both datasets with no observed false positives. Equal opportunity difference consistently proved the most sensitive metric, triggering first alerts at the lowest injection intensities and producing the largest peak deviations in both datasets. Predictive parity remained near baseline throughout all scenarios in both datasets, identifying as the least sensitive metric. The primary limitation surfaced in the bias-correction scenario, where the fixed threshold of 0.05 produced a six-batch detection latency gap between the two datasets. This suggests the pipeline would benefit from threshold scaling relative to each metric's baseline magnitude, a modification prioritized in future work.

VIII. TRADEOFFS & FUTURE MODIFICATIONS

The framework of this pipeline operates within a statistical margin of uncertainty, with its success dependent on the quality of human oversight. Such constraints mirror real-world roadblocks most developers face, and create an opportunity for more targeted technical and ethical improvements. The following limitations reflect such boundaries, while the proposed modifications serve as planned solutions.

The main technical limitation lies in the pipeline's alert threshold, fixed at an absolute value of 0.05 across all metrics. This value was manually selected based on natural variation observed during baseline training, with its deliberately low value increasing sensitivity to early drift. A more robust approach could determine thresholds dynamically through standard deviation, though this remains a point of open research in MLOps with no clear industry standard.

Beyond threshold sensitivity, the pipeline's scope is deliberately designed for drift detection, not correction. Without human oversight, automated correction poses as significant an ethical risk as undetected bias, as an unknowingly biased algorithm retraining itself would compound rather than resolve degradation. The EU's Artificial Intelligence Act mandates third-party intervention for high-risk models, providing the basis for this pipeline's foundational design: to surface bias for human intervention clearly and accessibly.

Structural concerns are also considered, as the system is bounded within binary constraints. The encoding approach does not accommodate multi-class demographics or intersectional bias, and monitors only one protected attribute per run — mirroring one of the primary limitations for real-world application. The scope is deliberately binary, as its purpose is to prototype the need for dynamic bias monitoring rather than serve as a comprehensive fairness solution.

The following modifications are proposed by priority. First, dataset compatibility would be expanded and validated across additional domains. Second, integrating a fairness retraining pipeline would allow for automated drift correction, addressing the most technically demanding limitation. Third, widening protected attribute support to include gender orientations, multiracial identities, and disability statuses would require redefining metrics alongside non-binary encoding processes. Finally, monitoring the fairness pipeline itself would enforce self-consistency and ensure zero systemic bias emerges from the infrastructure designed to detect it.

IX. ETHICAL IMPLICATIONS

When these models drift, the fallout lands hardest on the applicant first, the stakeholder with the least agency, unaware of the historical and systemic biases that skewed the

predictions against them. The consequences are not abstract, as negative outputs determine far more than a statistic. The pipeline primarily tests income prediction and employment screening, but the same harm distribution appears across any binary classification scenario, making it non-domain-specific and much more difficult to detect.

Evaluating fairness only at pre-deployment does not predict how a model will behave as real-world data distributions shift over time. Bias compounds gradually, rarely impacting measurable accuracy until it has already affected thousands of decisions. Unlike syntax errors, fairness drift cannot be flagged without continuous surveillance, as the pattern does not statistically surface until harm is no longer hypothetical. Identifying degradation at the simulated batch level allows for intervention before the fallout scales.

This is not an argument against AI-based decision-making, as these tools are not inherently harmful. The failure lies in the data they are trained on and the methods they are maintained by. Many organizations monitor ML infrastructure only when accuracy drops, but accuracy is subjective to the data it is tested on. An accurate model that encodes bias cannot be considered fully accurate, and a fair model at deployment will not necessarily remain so, as fairness is dynamic, not static.

Accountability must also be considered. The 2024 EU AI Act [5] classifies high-risk ML applications and mandates human oversight, audit trails, and documentation to ensure models are not solely determining output. The United States Equal Employment Opportunity Commission similarly requires such models to meet criteria under existing anti-discrimination law. These frameworks drive meaningful progress, though enforcement has not yet fully caught up with deployment pace.

Finally, both datasets used in this pipeline produced at least one baseline metric exceeding the alert threshold before any bias was injected, not as model errors, but as reflections of patterns present in the underlying data. The Adult Census baseline

statistical parity of -0.3350 reflects historical income inequality between men and women present in the underlying census data. Optimizing for accuracy can conflict with statistical parity, a tension formally documented in both US and EU regulatory literature. Monitoring pipelines surface this underlying bias, giving developers the information needed to make deliberate and informed threshold decisions rather than defaulting to unchecked outputs.

Resolution cannot occur without knowledge. Applicants across all demographics deserve to be evaluated on qualifications, by models managed by humans. When a model drifts from baseline, liability falls on those who built and deployed it, and this pipeline serves as infrastructure for that accountability.

X. REFERENCES

- [1] <https://archive.ics.uci.edu/dataset/2/adult>
- [2] <https://www.kaggle.com/datasets/tawfikelmetwally/employee-dataset>
- [3] <https://news.stanford.edu/stories/2025/10/ai-llms-age-bias-older-working-women-research>
- [4] <https://www.nature.com/articles/s41598-024-64210-5>
- [5] <https://artificialintelligenceact.eu/article/6/>
- [6] <https://plato.stanford.edu/entries/algorithmic-fairness/>
- [7] <https://www.python.org/doc/>
- [8] <https://docs.python.org/3/library/sqlite3.html#sqlite3-tutorial>
- [9] <https://www.geeksforgeeks.org/python/python-sqlite/>
- [10] <https://christophergs.com/machine%20learning/2020/03/14/how-to-monitor-machine-learning-models/>
- [11] <https://www.geeksforgeeks.org/machine-learning/ensuring-fairness-in-machine-learning-algorithms/>
- [12] <https://github.com/fairlearn/fairlearn>
- [13] <https://machinelearningmastery.com/statistical-hypothesis-tests-in-python-cheat-sheet/>
- [14] <https://machinelearningmastery.com/statistical-significance-tests-for-comparing-machine-learning-algorithms/>